CHAPTER 1

DESIGNING A CHESS GAME

1 Getting Ready: The Chess Game 2
2 Requirements for the Chess Game 7
3 Use Case Diagram for the Chess Game 9
4 Class Diagram for the Chess Game 13
5 Activity Diagram for the Chess Game 21
6 Code for the Chess Game . . 21
7 Summary: Designing a Chess Game 30
8 Key Insights and Tips: Designing a Chess Game 34

This chapter shows the design of a chess game, including requirements, use case, class, activity diagrams, and implementation code to develop the game.

1 Getting Ready: The Chess Game

1.1 Problem definition

Chess is a strategy board game for two players on a checkered board of 64 squares in an 8x8 grid. Each player starts with 16 pieces: one king, one queen, two rooks, two knights, two bishops, and eight pawns. The goal is to checkmate the opponent's king—this occurs when the king is in a position to be captured (**check**) and there is no way to move the king out of capture (**checkmate**).
Each piece has its unique movements. The rook can move horizontally or vertically, the knight can move in an L-shape position, the bishop can move diagonally, the queen can move in any direction, and the king can move one square in any direction. Pawns have the most complex movement rules: they can move forward one square, capture diagonally, and are subject to special rules such as a two-square advance on their first move, **en passant**, and **promotion**. Most pieces, except for the knight, cannot move through or over others. The game also includes special rules, such as **castling** for the king and the rook.
A chess game can end in a win, a loss, or a draw. Draws can result from stalemate, threefold repetition, the 50-move rule, insufficient material, mutual agreement, or other scenarios like resignation and forfeiture.
In this LLD interview case study, you'll focus on:

- Digitally modeling the rules, state, and movements of chess pieces on a virtual board, including special rules such as castling, en passant, and pawn promotion.
- Implementing turn-based gameplay, including legal moves, checks, checkmates, and various types of draws.
- Handling edge cases such as stalemate, threefold repetition, insufficient material, resignation, and forfeiture.
- The system will keep a complete record of all moves made during the game.
- Players cannot retract or undo moves once played.

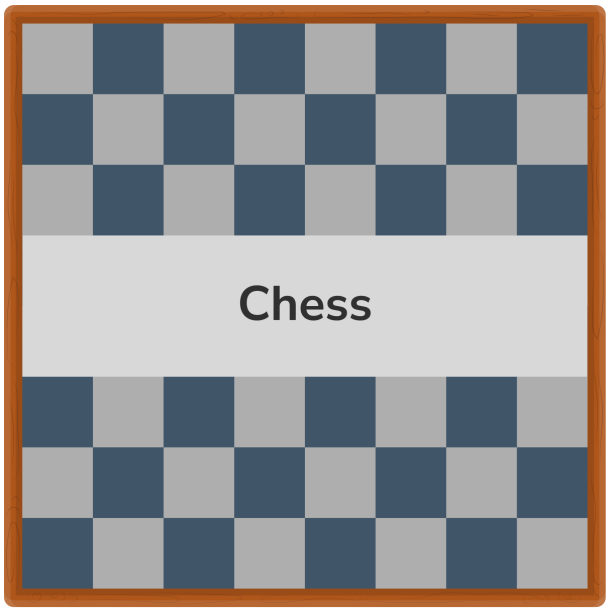
Note: Numerous variations of the game of chess are played globally. In this design problem, we'll create a digital version of the standard two-player chess game, faithfully implementing the official rules and recognized situations.

1.2 Expectations from the interviewee

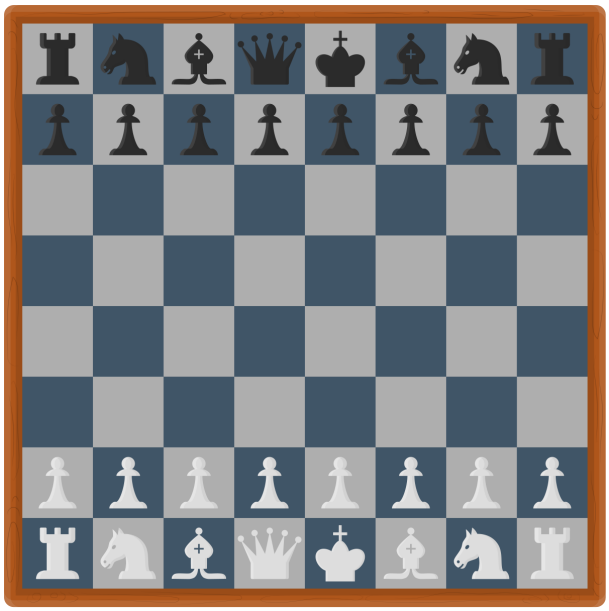
There are several components in a chess game, each with specific constraints and requirements. The following provides an overview of some of the main expectations that the interviewer will want to hear you discuss in more detail during the interview.

Chess pieces

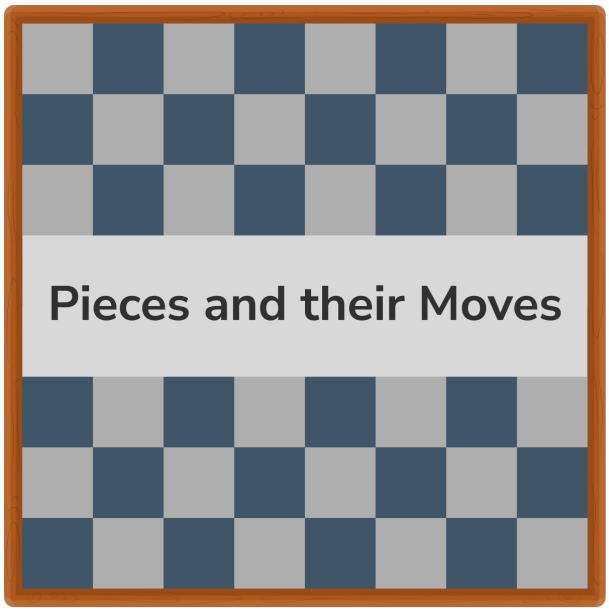
The chess pieces have their own specifications according to the game's rules that need to be clarified by the interviewer. Therefore, you may ask the following questions:



(a) An online chess game

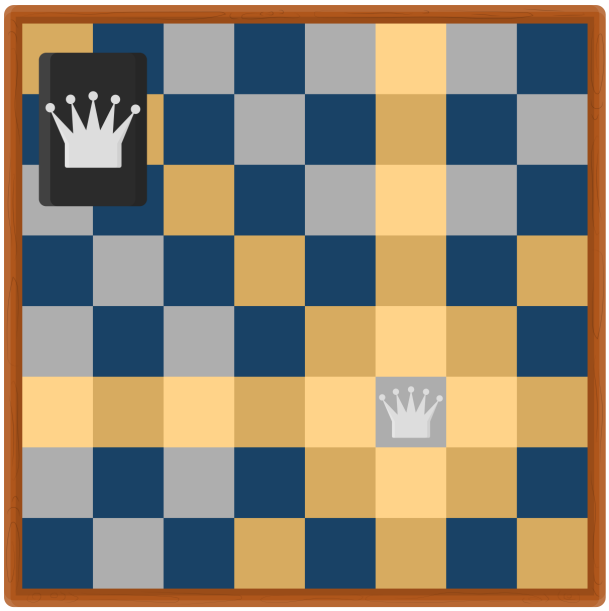


(b) The initial state of the chessboard before starting the game

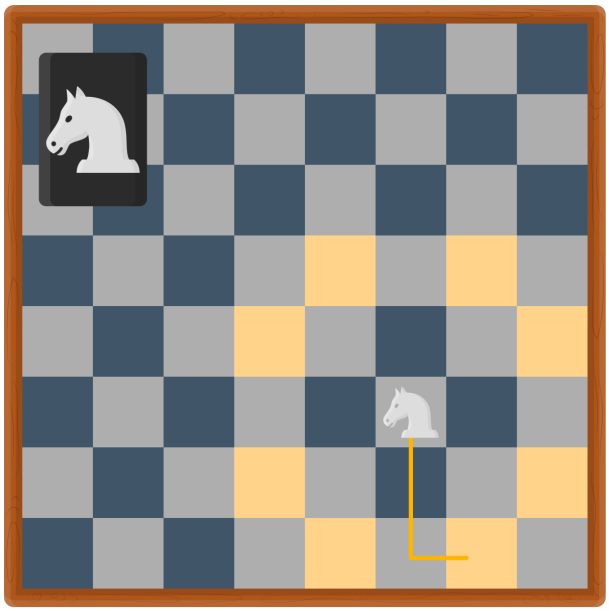


(a) Chess pieces and their moves

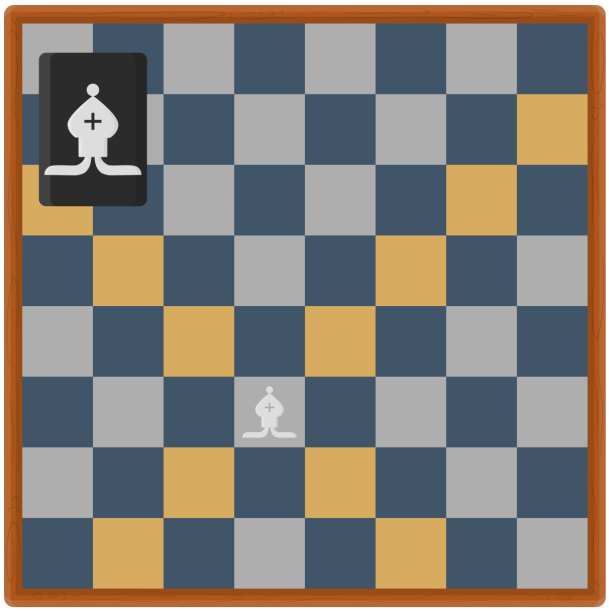
(b) The "king" piece and its moves



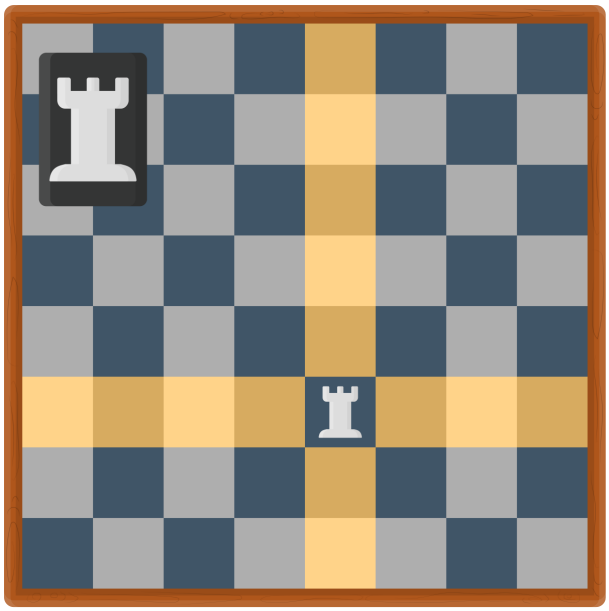
(a) The "queen" piece and its moves



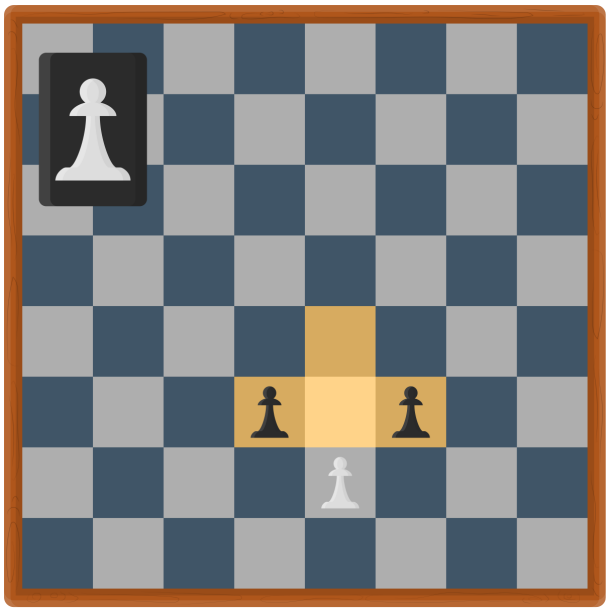
(b) The "knight" piece and its moves



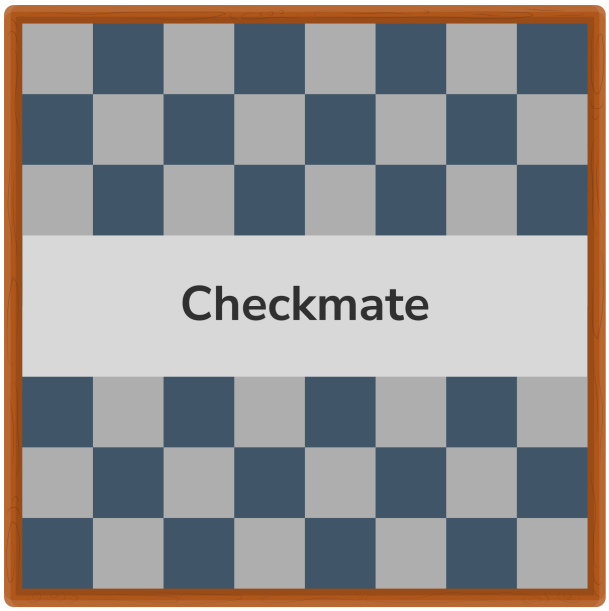
(a) The "bishop" piece and its moves



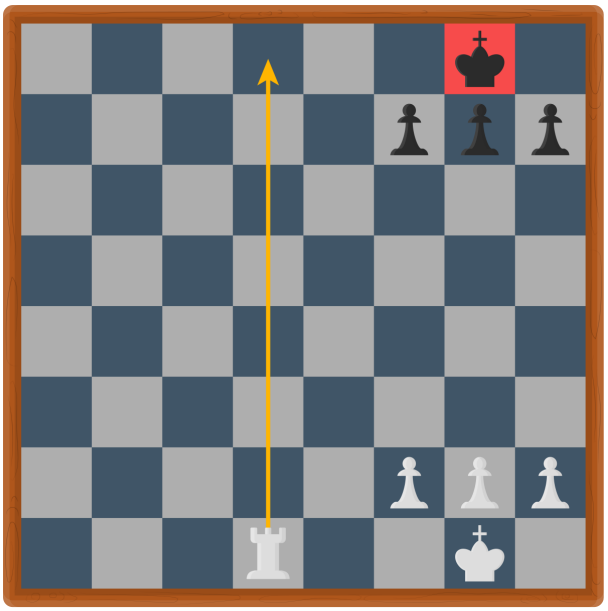
(b) The "rook" piece and its moves



(a) The “pawn” piece and its moves



(b) Checkmate in the game



(a) Demonstration of a scenario where a checkmate has occurred

1. How many chess pieces are there in the game?
2. What are the different chess pieces, and what are their respective moves?
3. Which piece is the weakest in chess?
4. Which piece is the strongest in chess?

Gameplay

In chess, the two opposing teams try to achieve the state of checkmate to win the game, or they face some special cases, like a stalemate, that need to be clarified by the interviewer. You may ask the following questions regarding them:

1. Which player takes the first turn?
2. What are the rules of the game?
3. What is a checkmate?
4. How does a stalemate happen?
5. Can a player forfeit/resign from the game?

1.3 Design approach

We will design this chess game system using a bottom-up approach. For this purpose, we will follow these steps:

- First, we'll define the core entities: `ChessPiece`, `BoardSquare`, and `Player`.
- Next, we'll model composite components like `ChessBoard`, `Move`, and `Game`, capturing piece movement, board state, and game progression.
- We'll ensure the design supports the enforcement of chess rules, detection of check, checkmate, and draw conditions, and handles move validation and player interactions.
- The system will be scalable for future extensions such as move history, undo/redo, and online multiplayer features.

This design approach will address edge cases and adhere to SOLID principles, making the system easy to extend, maintain, and test. Diagrams and code will illustrate major workflows and class structures.

1.4 Design pattern

During an interview, discussing the design patterns that the chess game falls under is always a good practice. Stating the design patterns gives the interviewer a positive impression and shows that the interviewee is well-versed in the advanced concepts of object-oriented design. Try to answer the following question. If you are not familiar with design patterns, don't worry! You can learn about them by asking, "Define design patterns."
Try to answer the following question. If you are not familiar with design patterns, don't worry! You can learn about them by asking questions like, "Define design patterns."

Question:

Which design pattern(s) should be used to design a Chess game? Please elaborate on your choice(s).

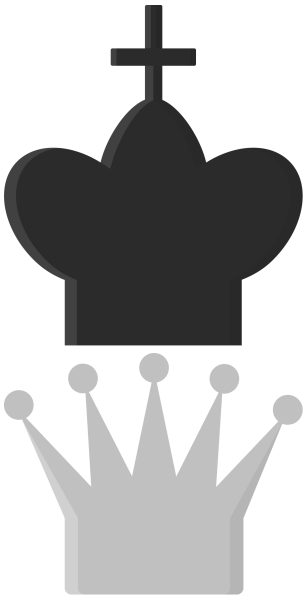
Let's explore the requirements of the online chess game in the next lesson.

2 Requirements for the Chess Game

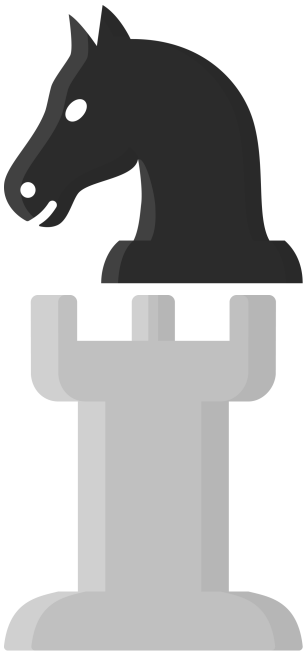
This lesson outlines the functional and operational requirements for an online chess game. Identifying and understanding requirements is essential to defining the system's scope and ensuring a robust, user-friendly design.
We'll use the notational convention to identify each requirement with a unique label "Rn," where "R" is short for requirement and "n" is a natural number.

2.1 Requirement collection

- **R1:** The system must enable two players (human or machine) to play chess online, supporting real-time, turn-based multiplayer gameplay.
- **R2:** The chess game must strictly enforce the official rules of international (standard) chess.
- **R3:** At the beginning of each game, players will be randomly assigned white or black.
- **R4:** Each player starts with 16 pieces: eight pawns, two rooks, two knights, two bishops, one queen, and one king, placed in the official starting positions on the 8x8 chessboard.



- **R5:** The player with the white pieces always makes the first move.
- **R6:** Players may not retract or undo their moves once made. (No "take-back" or undo functionality.)
- **R8:** The game must support all official ways a chess game can end, including:
  - Checkmate (a player's king is under attack and cannot escape)
  - Stalemate (no legal moves, and the king is not in check)
  - Draw (mutual agreement, threefold repetition, 50-move rule, or insufficient material)
  - Resignation (a player concedes defeat)
  - Forfeiture (a player does not show up or abandons the game)
- **R9:** According to international chess rules, the system must support special moves such as castling, en passant, and pawn promotion.



2.2 International chess rules

Rules for pieces

The following table represents the basic rules of chess:

Rules for situations

The following table represents certain situations we might face while playing chess:

We've identified our requirements for the problem. The next lesson will define different use cases for the online chess game design.

Figure 1.7: White pawn takes the first move

Piece	**Movement Rules**
King	Moves one square in any direction (horizontally, vertically, or diagonally). May perform castling once per game with a rook, if neither piece has moved, no pieces are between them, and the king does not move through, into, or out of check.
Queen	Moves any number of squares vertically, horizontally, or diagonally. Cannot jump over other pieces.
Rook	Moves any number of squares vertically or horizontally. Cannot jump over other pieces. Participates in castling with the king.
Bishop	Moves any number of squares diagonally. Cannot jump over other pieces.
Knight	Moves in an "L-shape": two squares in one direction (horizontal or vertical), then one square perpendicular. Can jump over other pieces.
Pawn	Moves forward one square. On its first move, it may move forward two squares. Captures one square diagonally forward. Can perform en passant capture. Promotes to queen, rook, bishop, or knight upon reaching the last rank.

Table 1.1: Chess Pieces and Their Movement Rules

3 Use Case Diagram for the Chess Game

In this lesson, we'll build the use case diagram for the online chess game system and understand the relationship between its main actor and core functions. First , let's define the

Situation	Description / Rule
Check	The king is under threat of capture. The player must make a move to remove the check.
Checkmate	The king is in check and has no legal moves to escape. The game ends, and the opponent wins.
Stalemate	The player has no legal moves, and their king is not in check. The game ends in a draw.
Draw	Can occur due to stalemate, threefold repetition, the 50-move rule (50 moves by each player with no pawn move or capture), insufficient material, or mutual agreement.
Forfeiture	A player forfeits if they do not show up for the game or abandon it.
Resignation	A player may resign at any time, conceding the game to the opponent.
Castling	A special move involving the king and a rook, performed on either the kingside or queenside. Allowed only if: • Neither the king nor the chosen rook has moved. • There are no pieces between the king and the rook. • The king is not in check, does not move through a square under attack, and does not end up in check. • The king has not previously been in check. The king moves two squares toward the rook, and the rook moves to the square the king passed over.
En Passant	A pawn that advances two squares from its starting position may be captured "en passant" by an opposing pawn as if it had moved only one square. This capture must be made immediately on the next move.
Pawn Promotion	When a pawn reaches the farthest rank from its starting position, it is promoted to a queen, rook, bishop, or knight, as chosen by the player.

Table 1.2: Special Situations and Rules in Chess

different elements of our chess game, followed by the complete use case diagram of the system.

3.1 System

Our online chess game system manages a standard chess match between two players over an online platform, strictly following official chess rules.

3.2 Actors

Here's the main actor of our chess game.

Primary actor

- **Player:** The primary and only actor in this system. A player participates in the chess game by making moves, viewing the game state, and may end the game by resigning or forfeiting.

3.3 Use cases

In this section, we'll define the use cases for the chess game. We have listed the use cases according to their interactions with a particular actor.

Player

- **Start new game:** To initiate a new chess match as either white or black.
- **Make move:** To select and move a chess piece according to the official rules, including special moves (castling, en passant, pawn promotion).
- **View game state:** To view the current board position, move history, and game status at any point during the match.
- **Resign/Forfeit game:** To voluntarily end the game before its conclusion, conceding victory to the opponent.

System

- **Validate move:** To validate that the move is legal according to all chess rules, including detection of check, checkmate, castling, en passant, and pawn promotion (Whenever a player attempts to make a move).
- **Declare results:** To check for endgame conditions such as checkmate, stalemate, draw (by threefold repetition, 50-move rule, or insufficient material), resignation, or forfeiture, and declare the appropriate result (after every move).

3.4 Relationships

This section describes the relationships between and among actors and their use cases.

Include

- "Make move" *includes* "Validate move:"
  - Every time a player moves, the system must validate that move according to chess rules (including check, checkmate, stalemate, en passant, castling, and promotion).

- "Validate move" *includes* "Declare results:"
  - After each move is validated, the system checks if the game has reached an end condition (checkmate, stalemate, draw, etc.) and declares the result if appropriate.
- "Resign/Forfeit game" *includes* "Declare results:"
  - When a player resigns or forfeits, the game ends immediately, and the system must declare the result (the opponent wins by resignation or forfeiture).

Associations

The table below shows the association relationship between actors and their use cases.

Player	**System**
Start new game	Validate move
Make move	Declare results
View game state	
Resign/Forfeit game	

3.5 Use case diagram

Here is the use case diagram of the chess game:

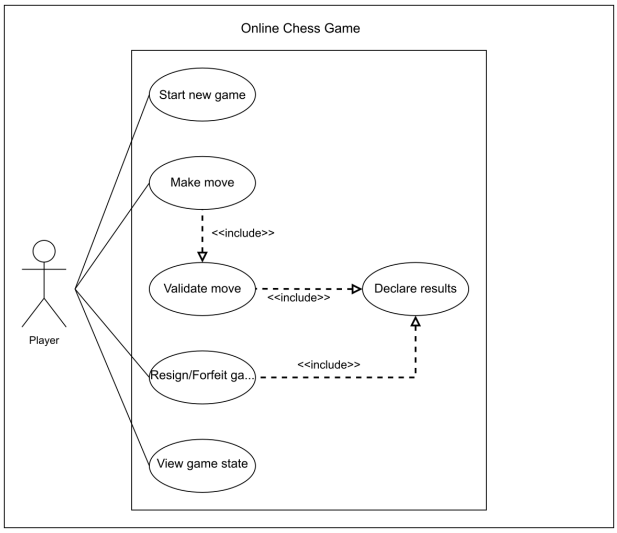


Figure 1.8: The use case diagram of a chess game

In the next lesson, we'll discuss the class diagram with a detailed explanation of all classes and their relationship with each other.

4 Class Diagram for the Chess Game

In this lesson, we'll create the class diagram for the online chess game. We 'll follow a bottom-up approach: design the simplest components and build up to the core game classes, showing relationships and responsibilities throughout.

4.1 Components of chess

As mentioned earlier, we'll follow the bottom-up approach to designing a class diagram for the chess game.

Box

A `Box` represents a position on the 8x8 chessboard, defined by a row and column (0–7). It may either be empty or occupied by a chess piece.

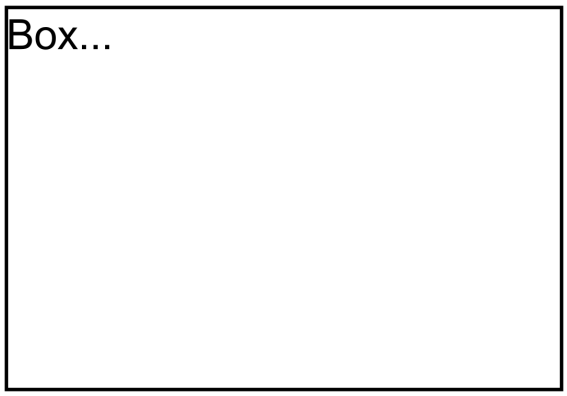


Figure 1.9: The class diagram of the Box class

Chessboard

The `Chessboard` models the 8x8 grid of boxes. It maintains the game's current state, including all pieces and their positions. The board is responsible for resetting and updating itself according to the moves played.

Piece

A `Piece` represents any chess piece (king, queen, rook, bishop, knight, or pawn) with a color (black or white) and an "alive" or "captured" status.

- `King`, `Queen`, `Rook`, `Bishop`, `Knight`, and `Pawn` are subclasses of `Piece`, each implementing its specific movement logic and special rules (e.g., castling for `King`/`Rook`, en passant, and promotion for `Pawn`).

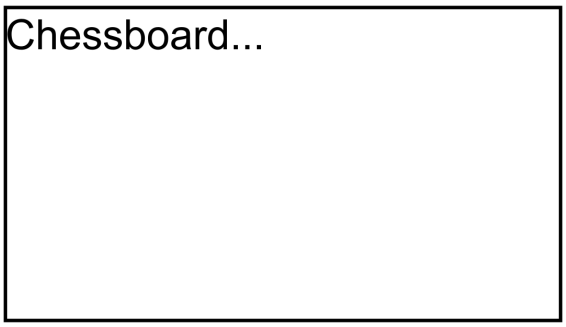


Figure 1.10: The class diagram of the Chessboard class

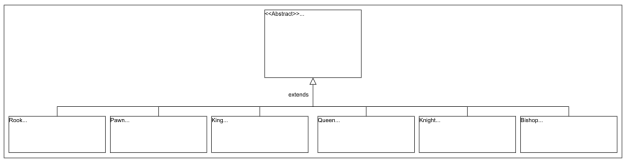


Figure 1.11: The class diagram of the Piece and its derived classes

R4: At the start of the game, each player will have eight pawns, two rooks, two bishops, two knights, one queen, and one king on the board.

Move

A `Move` represents the transfer of a piece from one box to another, possibly capturing an opponent's piece. The `Move` class tracks information such as the source and destination boxes, the moved piece, any captured piece, and whether the move resulted in special actions (e.g., castling, promotion).

R5: The player with the white pieces will make the first move.

R6: Once a move has been made, a player cannot retract or undo it.

Player

The `Player` represents one of the two participants in the game, associated with a color (white or black). The `Player` class is responsible for making moves and can resign or forfeit the game. Each player can review the game state and history.

R1: This system enables #key# multiplayer: A game that can be against the CPU or an online opponent. #key# in a game of chess via an online platform.

Chess move controller

The `ChessMoveController` encapsulates the logic for validating moves according to chess rules. It checks if a move is legal, applies it if valid, and communicates with the game to update the state. It handles rule enforcement, including check, checkmate, stalemate, castling, en passant, and promotion.

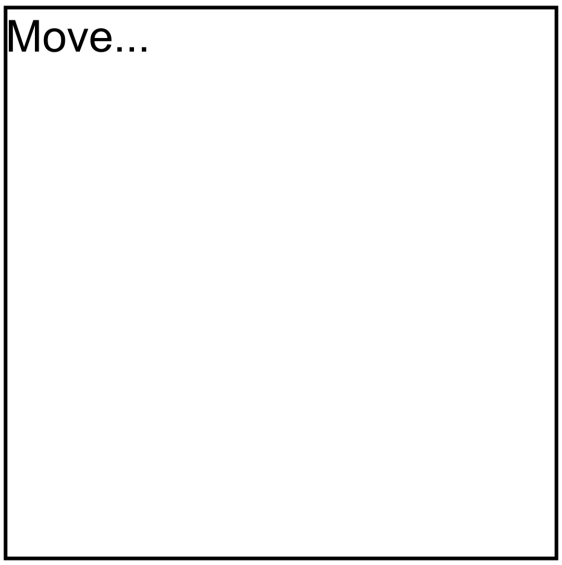


Figure 1.12: The class diagram of the Move class

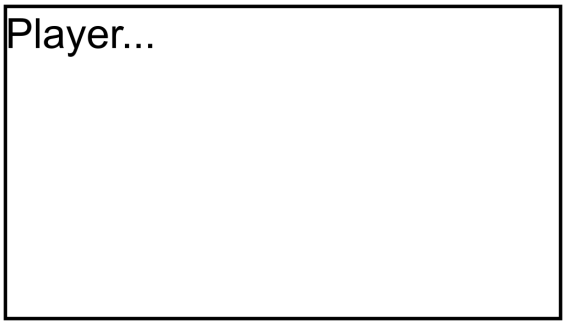


Figure 1.13: The class diagram of Player and its derived classes

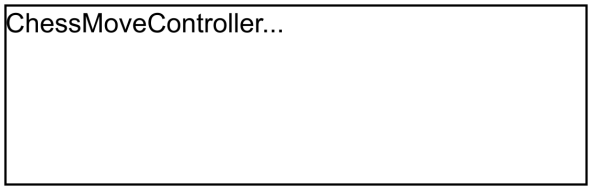


Figure 1.14: The class diagram of the ChessMoveController class

R2: The game will be played according to the official rules of an international chess game.

Chess game view

The `ChessGameView` is responsible for presenting the game's current state—board, moves, and status—to the players (e.g., through a UI or console output).

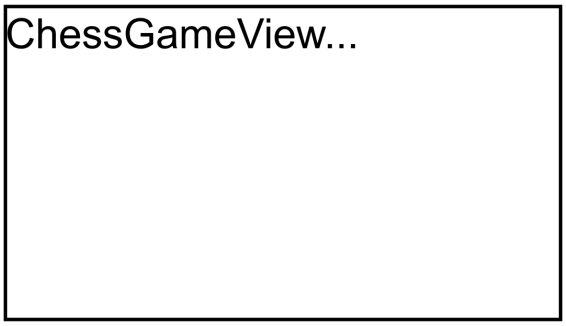


Figure 1.15: The class diagram of the ChessGameView class

Chess game

The `ChessGame` coordinates the gameplay, tracking both players, the current board, move history, and the game's status (in progress, checkmate, draw, etc.). It processes player turns, applies moves, and determines game outcomes.

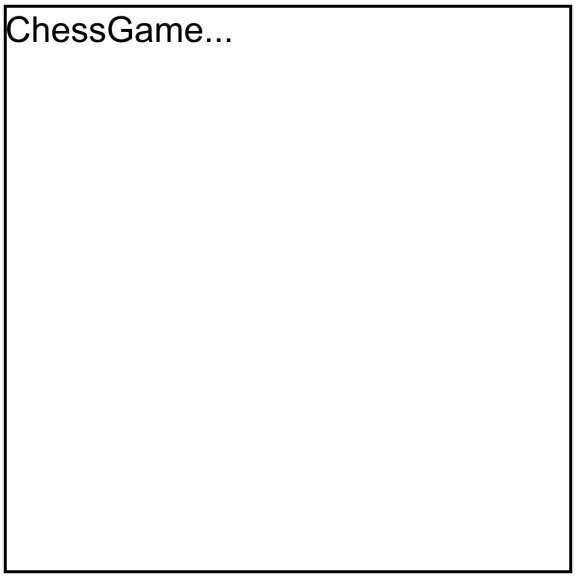


Figure 1.16: The ChessGame class

Enumerations and custom data types

The enumerations and custom data types required in the chess game design problem are listed below:

- **GameStatus:** Tracks the current game state (in progress, white wins, black wins, draw, etc.).
- **MoveType:** Enum for regular, castling, en passant, promotion, etc.

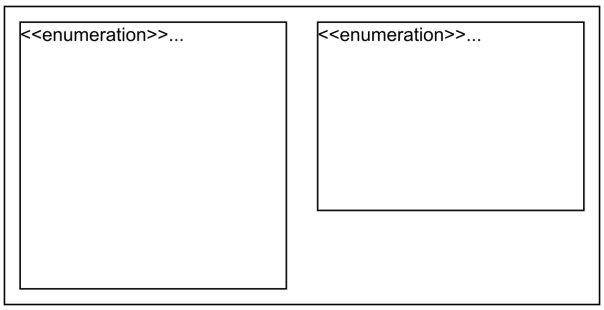


Figure 1.17: The class diagram of the Person class

4.2 Relationship between the classes

Now, we'll discuss the relationships between the classes we have defined above for our chess game design.

Association

The class diagram has the following association relationships:

- The `Move` class has a one-way association with `Player`, `Piece`, and `Box` (each move is linked to the player making the move, the piece moved, the captured piece (if any), and the start/end boxes).
- The `ChessGame` class has a one-way association with `ChessMoveController` (for move validation).
- The `ChessGame` class has a one-way association with `ChessGameView` (for updating the view/display).
- The `ChessMoveController` class has a one-way association with `ChessGame` (to access the game state/board as needed).
- The `ChessGameView` class has a one-way association with `ChessGame` or `Chessboard` (to display the current game state).

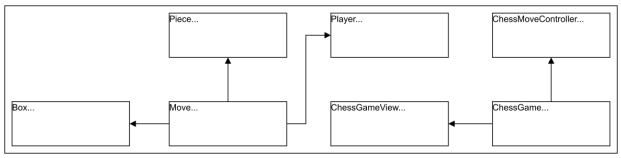


Figure 1.18: The association relationship between classes

Aggregation

The class diagram has the following aggregation relationships:

- The `ChessGame` class aggregates two `Player` objects (white and black).
  - (Players are part of a game but exist independently and can participate in other games.)

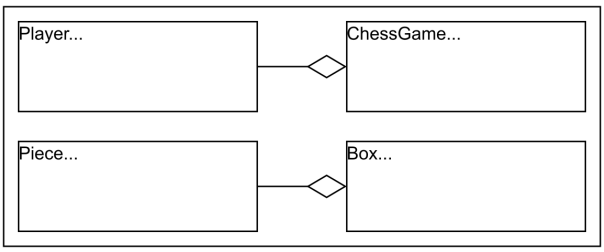


Figure 1.19: The aggregation relationship between classes

Composition

The class diagram has the following composition relationships.

- The `ChessGame` class is composed of a `Chessboard` and a collection of `Move` objects (the game controls their lifecycle).
- The `Chessboard` class is composed of 64 `Box` objects (boxes only exist as part of a board).
- Each `Box` may be composed of a `Piece` (a piece lives in a box; if removed from the board, it's no longer in play).

Inheritance

The following classes demonstrate an inheritance relationship:

- The `King`, `Queen`, `Rook`, `Bishop`, `Knight`, and `Pawn` classes extend the `Piece` class.

  Note: We have already discussed the inheritance relationship between classes in the component section above one by one.

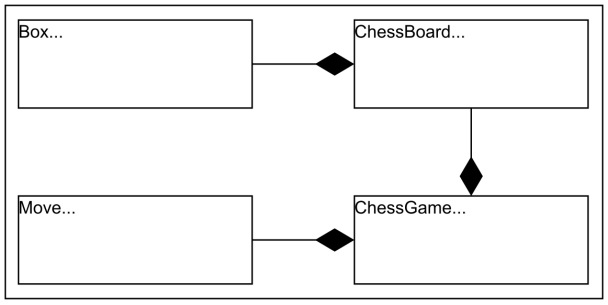


Figure 1.20: The composition relationship between classes

4.3 Class diagram for the chess game

This section outlines the multiplicity (cardinality) relationships between the main classes in our chess game. We explain the allowed number of instances on each side for each relationship and the real-world or design rationale behind the connection. Understanding these relationships is key to modeling how different entities interact and collaborate to support key workflows in the system.

Source	**Target**	**Multiplicity**	**Reason**
ChessGame	ChessBoard	1–1	Each game has exactly one chessboard.
ChessGame	Player	1–2	Each game has two players (white and black).
ChessGame	Move	1–0..*	Each game can have zero or more moves (move history).
ChessGame	ChessMoveController	1–1	Each game uses one move controller.
ChessGame	ChessGameView	1–1	Each game uses one view to display the game state.
ChessBoard	Box	1–64	Each board consists of exactly 64 boxes (8×8 grid).
Box	Piece	0..1–0..1	Each box may contain at most one piece, and a piece is in at most one box at a time.
Move	Player	1–1	Each move is made by exactly one player.
Move	Box	1–2	Each move is associated with a start box and an end box.
Move	Piece	1..2–1	Each move references one piece moved and possibly one piece captured.

Table 1.3: Entity Relationships and Multiplicities in a Chess Game

Here's the complete class diagram for our chess game:

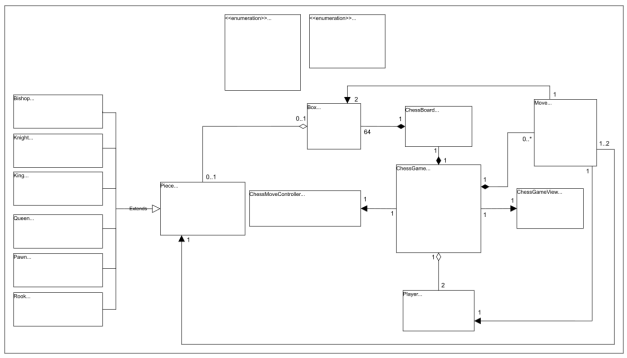


Figure 1.21: The class diagram of the chess game

4.4 Design pattern

The following design patterns have been used in the class diagram:

- **Singleton design pattern:** This pattern ensures the existence of a single instance of the chessboard at a given moment due to the shared nature of the chessboard as a resource. Multiple instances can cause the game state to become inconsistent.

- **Command design pattern:** This pattern encapsulates the move logic for each chess piece. Each piece has its own implementation of the move command, which allows it to move according to the rules defined for it. For example, the knight moves in an L-shape pattern, or the rook can move only horizontally or vertically on any number of boxes.

The following design patterns can also be used to design chess:

- The Iterator design pattern would enable the game to move sequentially by allowing the pieces to behave uniformly, where the user does not need to know the specifications or underlying logic behind the moves of the pieces.

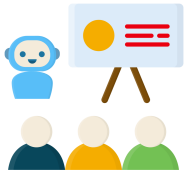
- The State design pattern ensures the encapsulation of each piece's state logic since all the chess pieces have their own respective implementations of checkmate states, which cause them to behave differently depending on the situation.

- The Observer design pattern enables the chess pieces to act as observers when the chessboard is the subject. As soon as the board's state changes, the pieces are notified to adapt accordingly, decoupling them from the board.

4.5 AI-powered trainer

At this stage, everything should be clear. If you encounter any confusion or ambiguity, feel free to utilize the interactive AI-enabled widget below to seek clarification. This tool is designed to help you strengthen your understanding of the concepts.

We have completed the class diagram of the chess game according to the requirements. In the next lesson, let's design its activity diagram.



5 Activity Diagram for the Chess Game

An activity diagram is a great way to visualize the flow of messages from one activity to another in the system. We can create different activity diagrams for our chess game. In this lesson, we will create an activity diagram to show how to play chess.

5.1 Chess game

The following states and actions will be involved in this activity diagram.

States

- **Initial state:** The player makes the first move.
- **Final state:** Any of the game-over conditions is met.

Actions

The player initiates a new game. The board appears, and the player takes a turn. The system validates each move and then checks if any game-over conditions (such as checkmate, resignation, forfeit, or draw) have been met. If the game is not over, play continues with the next turn. This process repeats until the system detects a game-over condition and automatically declares the result.
Based on this flow, the activity diagram of the chess game is given below.
We've looked at the activity diagram for our chess game. In the next lesson, we will present the code for our designed classes in some of the most popular programming languages.

We've looked at the activity diagram of our chess game. In the next lesson, we will present the code for our designed classes in some of the most popular languages.

6 Code for the Chess Game

Using various UML diagrams, we've covered different aspects of the chess game and observed the attributes of the problem. Let 's now explore the more practical side of things, where we will work on implementing the chess game using multiple languages. This is usually the last step in an object-oriented design interview process.
We have chosen the following languages to write the skeleton code of the different classes present in the chess game:

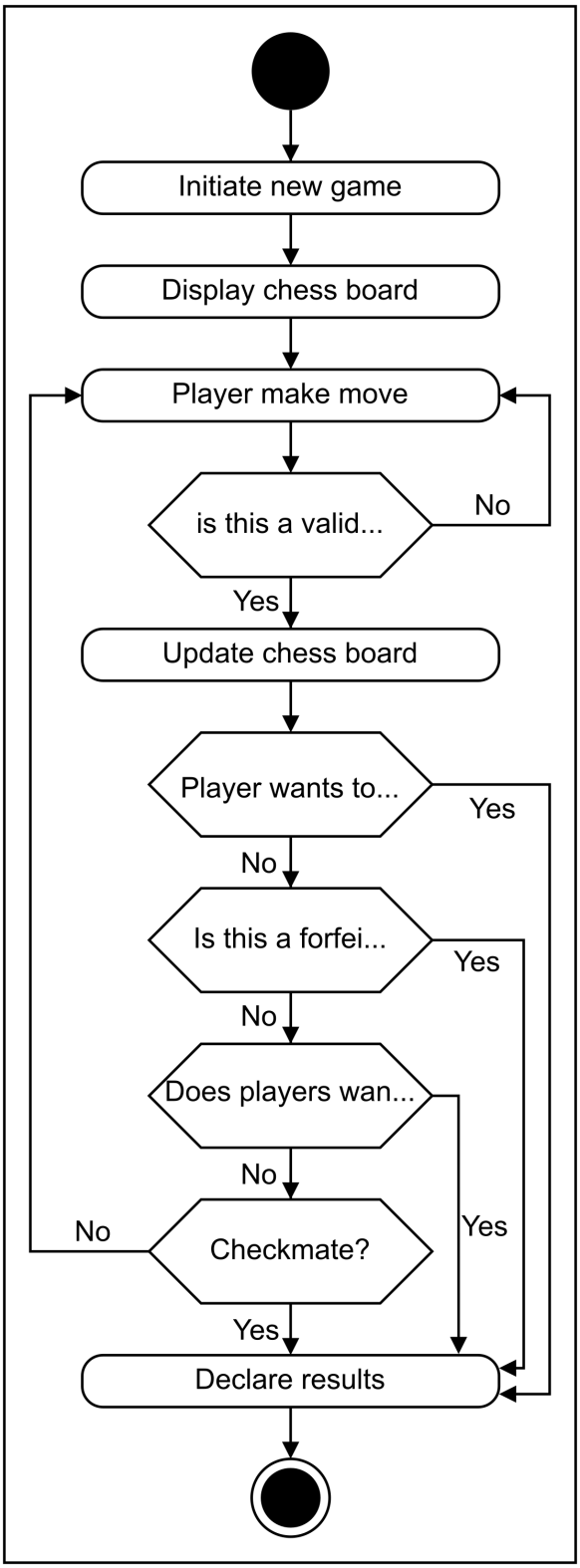


Figure 1.22: The activity diagram for an online chess game

- Java
- C#
- Python
- C++
- JavaScript

If you want to skip directly to the implementation and see the complete workflow, simply scroll down or click **Executable Code: Chess Game**.

6.1 Chess game classes

This section will provide the skeleton code of the classes designed in the class diagram lesson.

> **Note:** For simplicity, we are not defining getter and setter functions. The reader can assume that all class attributes are private, accessed through their respective public getter methods, and modified only through their public method functions.

Enumerations and custom data type

The following code defines the main enumerations used in the chess game:

- `GameStatus`: Tracks the current state of the game (active, win, draw, resignation, etc.).
- `MoveType`: Represents the type of move (normal, castling, en passant, promotion).

> **Note:** JavaScript does not support enumerations, so we will use the `Object.freeze()` method as an alternative that freezes an object and prevents further modifications.

Definition of enums and custom data types

```
public enum GameStatus {
    ACTIVE,
    WHITE_WIN,
    BLACK_WIN,
    DRAW,
    STALEMATE,
    RESIGNATION,
    FORFEIT
}

public enum MoveType {
    NORMAL,
    CASTLING,
    EN_PASSANT,
    PROMOTION
```

```
}

public enum AccountStatus {
    ACTIVE,
    CLOSED,
    CANCELED,
    BLACKLISTED,
    NONE
}
```

Box and chessboard

- `Box`: Represents a square on the board and may hold a reference to a `Piece`.
- `Chessboard`: Contains the 8x8 array of boxes.

The definitions of these classes are provided below:

The Box and Chessboard classes

```
public class Box {
    private int x, y;
    private Piece piece;

    public Box(int x, int y, Piece piece) {
        this.x = x;
        this.y = y;
        this.piece = piece;
    }

    public int getX() { return x; }
    public int getY() { return y; }
    public Piece getPiece() { return piece; }
    public void setPiece(Piece piece) { this.piece = piece; }
}

public class Chessboard {
    private Box[][] boxes = new Box[8][8];

    public Chessboard() {
        for (int i = 0; i < 8; i++)
            for (int j = 0; j < 8; j++)
                boxes[i][j] = new Box(i, j, null);
    }

    public Box getBox(int x, int y) { return boxes[x][y]; }
    public Box[][] getBoxes() { return boxes; }
}
```

Piece

- `Piece`: An abstract class representing a chess piece.
- `King`, `Queen`, `Rook`, `Bishop`, `Knight`, `Pawn`: Subclasses of `Piece`.

The definitions of these classes are provided below:

The Piece class and its derived classes

```java
public abstract class Piece {
    private boolean white;
    private boolean killed = false;

    public Piece(boolean white) {
        this.white = white;
    }

    public boolean isWhite() { return white; }
    public boolean isKilled() { return killed; }
    public void setKilled(boolean killed) { this.killed = killed; }
}

public class King extends Piece {
    public King(boolean white) { super(white); }
}

public class Queen extends Piece {
    public Queen(boolean white) { super(white); }
}

public class Rook extends Piece {
    public Rook(boolean white) { super(white); }
}

public class Bishop extends Piece {
    public Bishop(boolean white) { super(white); }
}

public class Knight extends Piece {
    public Knight(boolean white) { super(white); }
}

public class Pawn extends Piece {
    public Pawn(boolean white) { super(white); }
}
```

Move

`Move`: Represents a player's move, including the start and end boxes, the piece moved, the captured piece (if any), and the player who made the move.
The definitions of these classes are provided below:

The Move class

```
public class Move {
    private Box start;
    private Box end;
    private Piece pieceMoved;
    private Piece pieceKilled;
    private Player player;
    private MoveType moveType;

    public Move(Box start, Box end, Piece pieceMoved, Piece pieceKilled, Player
        player, MoveType moveType) {
        this.start = start;
        this.end = end;
        this.pieceMoved = pieceMoved;
        this.pieceKilled = pieceKilled;
        this.player = player;
        this.moveType = moveType;
    }

    public Box getStart() { return start; }
    public Box getEnd() { return end; }
    public Piece getPieceMoved() { return pieceMoved; }
    public Piece getPieceKilled() { return pieceKilled; }
    public Player getPlayer() { return player; }
    public MoveType getMoveType() { return moveType; }
}
```

Player

`Player`: Represents a player, with name and side (white or black).
The definitions of these classes are provided below:

The Account class and its derived classes

```
public class Player {
    private String name;
    private boolean whiteSide;

    public Player(String name, boolean whiteSide) {
        this.name = name;
        this.whiteSide = whiteSide;
    }

    public String getName() { return name; }
    public boolean isWhiteSide() { return whiteSide; }
}
```

Chess move controller and the game view

- `ChessMoveController`: Responsible for validating moves.
- `ChessGameView`: Handles display and visualization of the game board and moves.

The definitions of these classes are provided below:

The ChessMoveController and ChessGameView classes

```
public class ChessMoveController {
    public ChessMoveController() {}

    // Future: add methods for move validation, etc.
}

public class ChessGameView {
    public ChessGameView() {}

    // Future: add methods for displaying the board/moves
}
```

Chess game

`ChessGame`: Orchestrates gameplay, manages the board, players, move history, and game status.

The definition of this class is provided below:

The ChessGame class

```
import java.util.*;

public class ChessGame {
    private Player[] players;
    private Chessboard board;
    private Player currentTurn;
    private GameStatus status;
    private List<Move> moveHistory;
    private ChessMoveController controller;
    private ChessGameView view;

    public ChessGame(Player white, Player black) {
        this.players = new Player[] { white, black };
        this.board = new Chessboard();
        this.currentTurn = white;
        this.status = GameStatus.ACTIVE;
        this.moveHistory = new ArrayList<>();
        this.controller = new ChessMoveController();
        this.view = new ChessGameView();
    }

    public Player[] getPlayers() { return players; }
```

```
    public Chessboard getBoard() { return board; }
    public Player getCurrentTurn() { return currentTurn; }
    public GameStatus getStatus() { return status; }
    public List<Move> getMoveHistory() { return moveHistory; }
    public ChessMoveController getController() { return controller; }
    public ChessGameView getView() { return view; }

    public void setCurrentTurn(Player player) { this.currentTurn = player; }
    public void setStatus(GameStatus status) { this.status = status; }
}
```

6.2 Executable code: Chess game

Below is a fully self-contained, runnable program in Java, C#, C++, Python, and JavaScript that demonstrates the core workflows of the Chess game. The system's main driver code resides in the `Driver.java`, `Driver.cs`, `Driver.py`, `Driver.cpp`, and `Driver.js` for all respective languages. You can click the "Run" button to execute the codes.

What this code shows

- **System initialization:** Sets up a chessboard, two players, and starts the game.
- **Scenario-based play:** Demonstrates moves for both players, including normal moves, captures, illegal move handling, and resignation.
- **Move log:** Prints a full record of all valid moves made during the demonstration.

Hint: Try customizing the code!
This code is designed for experimentation, not just reading. You can edit, extend, or modify any part of the example.

Executable code: Chess game

```
import java.util.*;

public class Driver {
    public static void main(String[] args) {
        Player white = new Player("Alice", true);
        Player black = new Player("Bob", false);
        ChessGame game = new ChessGame(white, black);

        // Sample game id
        System.out.println("Game ID: " + game.getGameId());

        game.getBoard().resetBoard();
        game.getView().showBoard(game.getBoard());

        // Scenario 1: White moves pawn e2 to e4
        playMove(game, white, "e2", "e4");
```

```
        // Scenario 2: Black moves pawn e7 to e5
        playMove(game, black, "e7", "e5");

        // Scenario 3: White moves Queen d1 to h5
        playMove(game, white, "d1", "h5");

        // Scenario 4: Black moves Knight b8 to c6
        playMove(game, black, "b8", "c6");

        // Scenario 5: White moves Bishop f1 to c4
        playMove(game, white, "f1", "c4");

        // Scenario 6: Black moves Knight g8 to f6
        playMove(game, black, "g8", "f6");

        // Scenario 7: White moves Queen h5 to f7 (Fool's Mate checkmate, if black
            played g7-g6 earlier)
        playMove(game, white, "h5", "f7");

        // Scenario 8: White tries an illegal move (move pawn e4 to e5 - but e4 is
            now empty)
        playMove(game, white, "e4", "e5");

        // Scenario 9: Black resigns
        game.setStatus(GameStatus.WHITE_WIN);
        System.out.println("
Bob (Black) resigns! Alice (White) wins!");

        // Print move log
        System.out.println("
Move log:");
        for (Move m : game.getMoveHistory()) {
            System.out.println(" " + m);
        }
    }

    // Helper to parse and play a move
    static void playMove(ChessGame game, Player player, String from, String to) {
        int sx = from.charAt(1) - '1', sy = from.charAt(0) - 'a';
        int ex = to.charAt(1) - '1', ey = to.charAt(0) - 'a';
        Box start = game.getBoard().getBox(sx, sy);
        Box end = game.getBoard().getBox(ex, ey);
        Piece piece = start.getPiece();
        if (piece == null) {
            System.out.println("
No piece at source (" + from + ") for " + player + "!");
            return;
        }
        if (piece.isWhite() != player.isWhiteSide()) {
            System.out.println("
Not your piece (" + from + ") for " + player + "!");
            return;
        }
        if (!game.getController().validateMove(piece, game.getBoard(), start, end))
            {
            System.out.println("
```

```
Invalid move for " + piece.getSymbol() + " from " + from + " to " + to + "!");
            return;
        }
        if (piece instanceof Pawn) {
            int promotionRow = piece.isWhite() ? 7 : 0;
            if (end.getX() == promotionRow) {
                // For demo: auto-promote to Queen
                end.setPiece(new Queen(piece.isWhite()));
                System.out.println(piece.isWhite() ? "White" : "Black" + " pawn
                    promoted to Queen at " +
                                (char)('a' + end.getY()) + (end.getX() + 1));
            }
        }
        Piece captured = end.getPiece();
        end.setPiece(piece);
        start.setPiece(null);
        Move move = new Move(start, end, piece, captured, player, MoveType.NORMAL);
        game.getMoveHistory().add(move);
        System.out.println("
" + player + " moves " + piece.getSymbol() + " from " + from + " to " + to +
            (captured != null ? " capturing " + captured.getSymbol() : ""));
        game.getView().showBoard(game.getBoard());
    }
}
```

6.3 Wrapping up

In this chapter, we've explored the complete design of the chess game. We 've also looked at how a basic chess game can be visualized using various UML diagrams and designed using object-oriented principles and design patterns.

7 Summary: Designing a Chess Game

Now that you've completed the chess game case study, let's take a moment to reflect on and consolidate what we've learned. We 'll revisit the key system requirements, identify the core classes along with their responsibilities and relationships, and highlight the major design principles applied. We 'll also examine how objects interact within the system and walk through the overall workflow to understand how the components come together to achieve the desired functionality.

7.1 Key requirements

The following are the essential requirements that define the functional scope and expected behavior of the online chess game:

1. Support two-player, real-time, turn-based chess gameplay online.

2. Enforce the official international chess rules throughout gameplay.
3. Randomly assign player colors (white or black) at the start of the game.
4. Initialize the game board with 16 pieces per player in standard positions.
5. The player with the white pieces always makes the first move.
6. Players are not allowed to undo or retract their moves.
7. Maintain a complete move history for every game session.
8. Recognize all valid game-ending scenarios: checkmate, stalemate, draw, resignation, and forfeiture.
9. Implement all special moves: castling, en passant, and pawn promotion.

7.2 System actors

This section outlines the main actor who interacts with the chess game system.

- **Player:** Participates in a chess match by starting games, making moves, reviewing game state, and ending games via resignation or forfeiture.

7.3 Important classes and relationships

The chess system has core and supporting classes that model pieces, gameplay flow, and game state.

7.4 Design approach

The system is designed using a bottom-up approach, beginning with low-level abstractions like `Box` and `Piece`, and progressively integrating them into higher-order components like `ChessBoard`, `Move`, and `ChessGame`.

Design principles

The system applies SOLID principles to maintain separation of concerns, enable flexibility, and ease maintenance:

- **Single Responsibility principle (SRP):** Each class handles one distinct responsibility, such as board management, move validation, or gameplay coordination.
- **Open/Closed principle (OCP):** The `Piece` class hierarchy and `MoveType` enum support easy extension of new logic without modifying existing code.
- **Liskov Substitution principle (LSP):** All subclasses of `Piece` (e.g., `Knight`, `Bishop`) can be used interchangeably where `Piece` is expected.

Class	Description	Important Relationships
Box	Represents a square on the 8×8 board.	Composed in ChessBoard; may contain a Piece.
ChessBoard	Manages the 64 Box instances and the current layout of the game.	Composed of Box; associated with ChessGame.
Piece	Base class for all chess pieces with color and status.	Inherited by King, Queen, Rook, Bishop, Knight, and Pawn; may exist in a Box.
Move	Captures details of a player's move (from-box, to-box, piece moved, etc.).	Associated with Player, Piece, and Box.
Player	Represents a user participating in the game.	Aggregated in ChessGame; associated with Move.
ChessMoveController	Validates and executes moves based on chess rules.	Associated with ChessGame.
ChessGameView	Displays the board and game status to players.	Associated with ChessGame.
ChessGame	Coordinates gameplay, turn-taking, game state, and results.	Composed of ChessBoard and Move; aggregates two Player instances.
GameStatus (Enum)	Indicates the current status: in progress, draw, white wins, black wins, etc.	Used by ChessGame.
MoveType (Enum)	Classifies move types: regular, castling, en passant, promotion.	Used by Move.

Table 1.4: Chess Game Classes and Their Relationships

- **Interface Segregation principle (ISP):** Components like `ChessMoveController` and `ChessGameView` focus on specialized interfaces and behaviors.
- **Dependency Inversion principle (DIP):** High-level modules such as `ChessGame` interact with abstractions (`Player`, `Move`) rather than concrete implementations.

Design patterns

Several design patterns strengthen the robustness and adaptability of the chess system:

- **Singleton pattern:** Ensures a single `ChessBoard` instance exists during gameplay to maintain consistency.
- **Command pattern:** Encapsulates move behavior within each `Piece` class for polymorphic execution.
- **Observer pattern:** Allows `Piece` objects to react to changes in the board state, especially relevant for check and checkmate detection.

- **State pattern:** Can handle player turns, move sequences, and transition between gameplay states.
- **Iterator pattern (recommended):** Useful for traversing valid moves or iterating through board tiles.

7.5 Object interactions

This section describes how key system components collaborate to handle game operations dynamically.

- **Start game:**
  - `Player` joins or initiates a new game.
  - `ChessGame` assigns colors, initializes the board, and sets the starting state.
- **Make move:**
  - `Player` selects a piece and a destination.
  - `ChessMoveController` validates the move against the rules.
  - If valid, `Move` is recorded and `ChessGame` updates the game state.
  - `ChessGameView` reflects the new state.
- **Check for endgame:**
  - After each move, `ChessGame` checks for checkmate, stalemate, draw, etc.
  - If any condition is met, `GameStatus` is updated.
- **Resign or forfeit:**
  - `Player` may resign or forfeit.
  - `ChessGame` marks the game over and declares the result.

7.6 System workflow

This section outlines the end-to-end flow of the game from player actions to system responses.

- **Initialize game:**
  - Two players are assigned colors, and pieces are positioned.
  - `ChessGame` sets the initial state and prompts the first move.
- **Gameplay loop:**

  - On each turn, a player makes a move.
  - `ChessMoveController` validates the move.
  - If valid, the board state is updated and the move is logged.
  - System checks for end conditions (checkmate, draw, resignation, etc.).

- **End game:**
  - If an end condition is met, `ChessGame` updates `GameStatus`.
  - `ChessGameView` displays the final board and outcome to players.

8 Key Insights and Tips: Designing a Chess Game

Congratulations on completing the case study for designing an online chess game. This lesson builds on your foundational knowledge by surfacing common design mistakes, providing targeted interview tips, and presenting a quick quiz to check your understanding. You 'll also explore related case studies, reinforcing key object-oriented concepts like inheritance, game state management, and modular design.

8.1 Common mistakes

Here are some common mistakes interviewees make when designing the chess game system:

- **Ignoring special rules like castling or en passant:** Failing to support nuanced mechanics like castling, en passant, or pawn promotion leads to an incomplete design. Ensure your `ChessMoveController` includes logic for handling these special rules.
- **Designing piece movements as procedural logic rather than polymorphic behavior:** Hardcoding all movement rules in one place violates object-oriented design. Instead, each `Piece` subclass should override the `canMove()` method to encapsulate its movement logic.
- **Do not separate game state from UI logic:** Mixing UI concerns with gameplay logic reduces modularity and testability. Keep the display logic in `ChessGameView` and the core game rules in `ChessGame` and `ChessMoveController`.
- **Not considering draw conditions:** Overlooking draw-by-stalemate, threefold repetition, or insufficient material weakens rule completeness. Track game state transitions and use `GameStatus` enums to detect draw situations.
- **Using multiple instances of the `ChessBoard`:** Allowing more than one board instance breaks game consistency. Apply the Singleton pattern to ensure a single `ChessBoard` object is shared across the game.

8.2 Key interview tips

Here are targeted tips to help you excel in chess game OOD questions:

Tip	Explanation
Separate concerns via MVC.	Modularity in game logic (Model), UI (View), and actions (Controller) is key to scalability and testability.
Design with extensibility in mind.	Support future additions like AI players or time controls by abstracting components cleanly.
Validate moves through a controller.	Centralize move validation in a ChessMoveController class rather than distributing logic across pieces or the board.
Capture game states with enums.	Use enums for GameStatus and AccountStatus to improve readability and reduce errors.
Use composition and aggregation appropriately.	Classes like ChessBoard should contain Box objects through composition, while ChessGame aggregates Player instances.

Table 1.5: Design Tips for a Chess Game System

8.3 Test your understanding

Assess your grasp of the design principles and structure for the chess game system:

Designing a Chess Game

Question 1: Why is the Singleton pattern used for the `ChessBoard` class in a chess game system?

- To ensure thread safety during gameplay
- To prevent multiple players from making moves
- **[CORRECT]** To guarantee a consistent, single shared board across the game life cycle
  Explanation: Singleton ensures only one shared `ChessBoard` instance exists, maintaining a consistent game state for both players throughout the game.
- To allow each player to have their own board view

Question 2: What is the best way to represent unique movement logic for each piece?

- **[CORRECT]** Switch-case logic in the controller

- Inheritance with overridden methods
  Explanation: Inheritance and polymorphism allow each piece to implement its specific movement logic.
- Enum flags in the `Piece` class
- Separate movement utility class

Question 3: What relationship best describes a `ChessBoard` having Box objects?

- Association
- Aggregation
- **[CORRECT]** Composition
  Explanation: Composition is a "part-of" relationship; a board is composed of 64 boxes.
- Inheritance

Question 4: Why is move validation handled in a separate class, `ChessMoveController`?

- To comply with the factory pattern
- To simplify the UI logic
- **[CORRECT]** To encapsulate rule logic and improve separation of concerns
  Explanation: Move validation is centralized to ensure clean separation of rules from UI and data layers.
- To reduce memory consumption

Question 5: Which design decision best enables the implementation of undo and redo in a chess game system?

- Tracking board state in a global variable.
- **[CORRECT]** Representing each move as a command object.
  Explanation: Using the Command pattern allows moves to be encapsulated as objects, making it easier to implement undo/redo functionality cleanly.
- Storing each player's move in a static class.
- Saving the UI output of each game state.

8.4 Related case studies

To further strengthen your understanding, explore these related OOD case studies:

- **Design a tic-tac-toe game:** A simpler board game ideal for practicing fundamental concepts like turn-taking, win/draw conditions, and board state tracking. It serves as a great stepping stone before designing full-scale games like chess.
- **Design a multiplayer card game (e.g., Uno):** Focuses on player interactions, game rules, deck management, and special action handling—parallels special chess rules.
- **Designing a jigsaw puzzle:** Emphasizes spatial arrangement, piece interaction, and grid modeling. While not turn-based, it shares structural modeling similarities with how pieces occupy cells (like chess pieces on boxes).
- **Designing a traffic light control system:** Models real-world concurrency, finite state machines, and timing-based transitions. It challenges candidates to handle multiple intersections, emergency overrides, and sequencing logic—great for testing clarity in state and event-driven design.
- **Designing a restaurant management system:** Though a non-game system, it teaches clear separation of concerns, command-driven flows, and role-based interactions—parallels that help organize controllers and user roles in a chess system.